

Universidad Nacional Autónoma de México

Facultad de Ingeniería



Sistemas Distribuidos

Semestre 2027 - 1

Laboratorio S8. Que la transferencia no deje dinero en el aire

Alumno: Fernández Herrera Mauricio

Equipo: Pumaverso Distribuido.

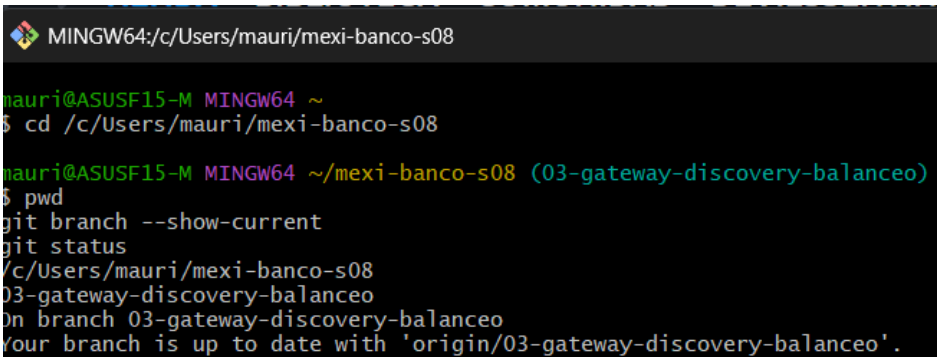
Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

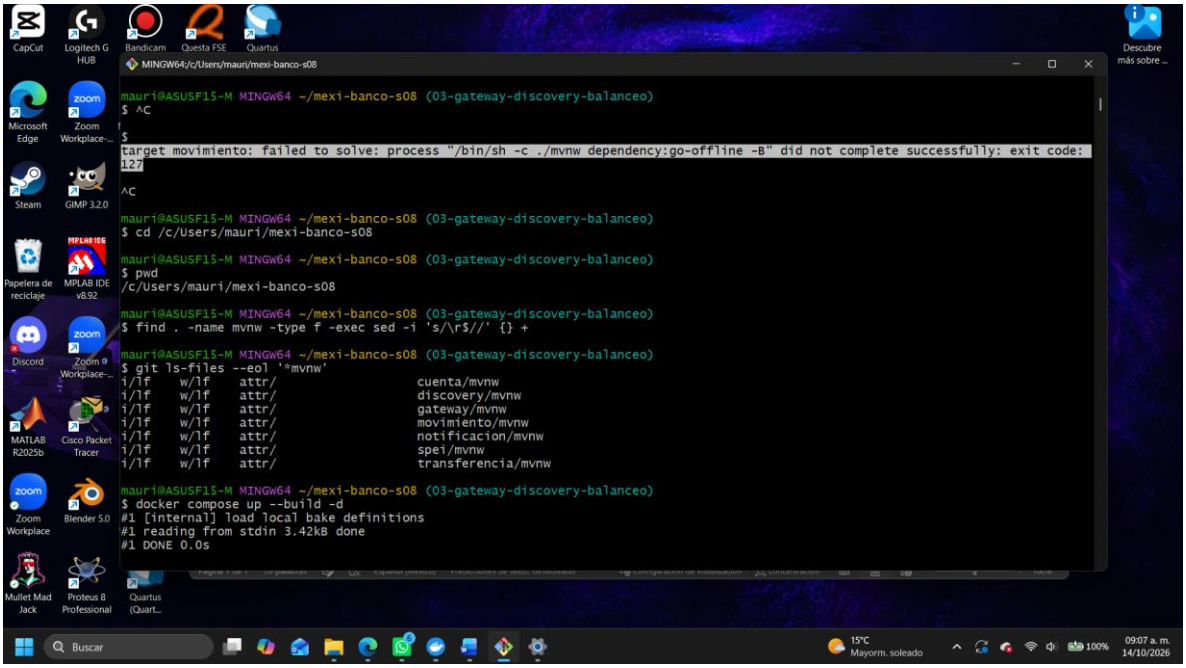
Entrega: 17/10/2026

1.- El punto de partida. 14/10/2026 – 8:58am.

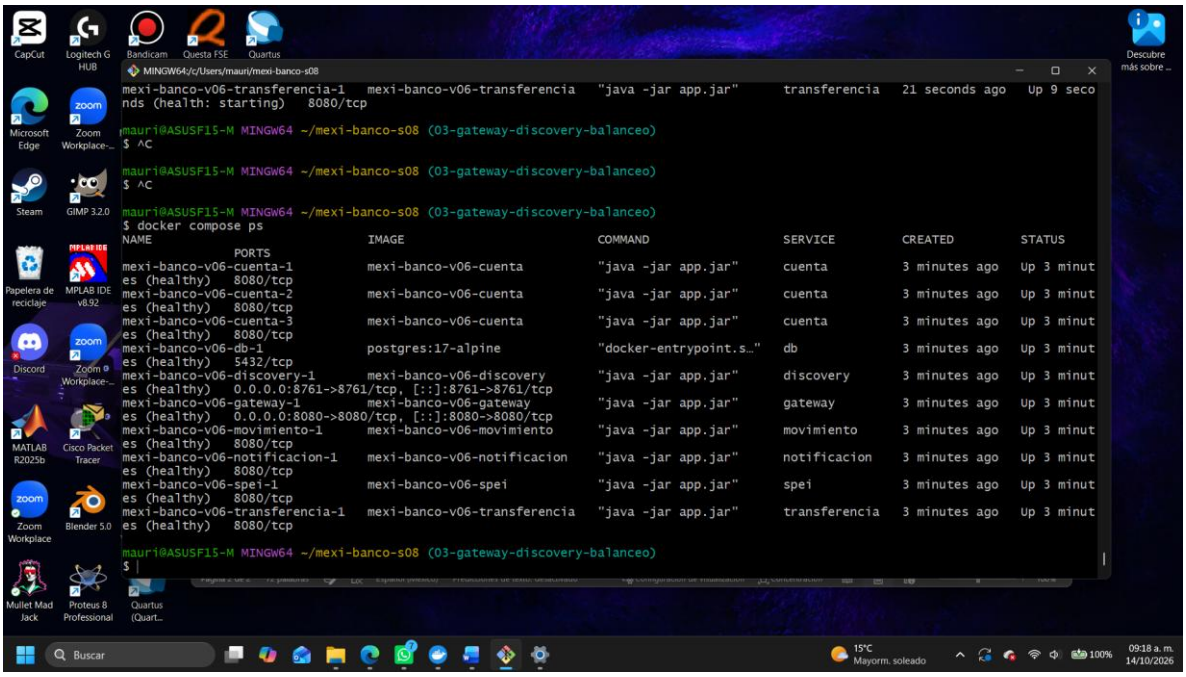
Nos situamos en la carpeta y rama necesarias para comenzar a trabajar tras haber clonado el repositorio de mexi-banco.



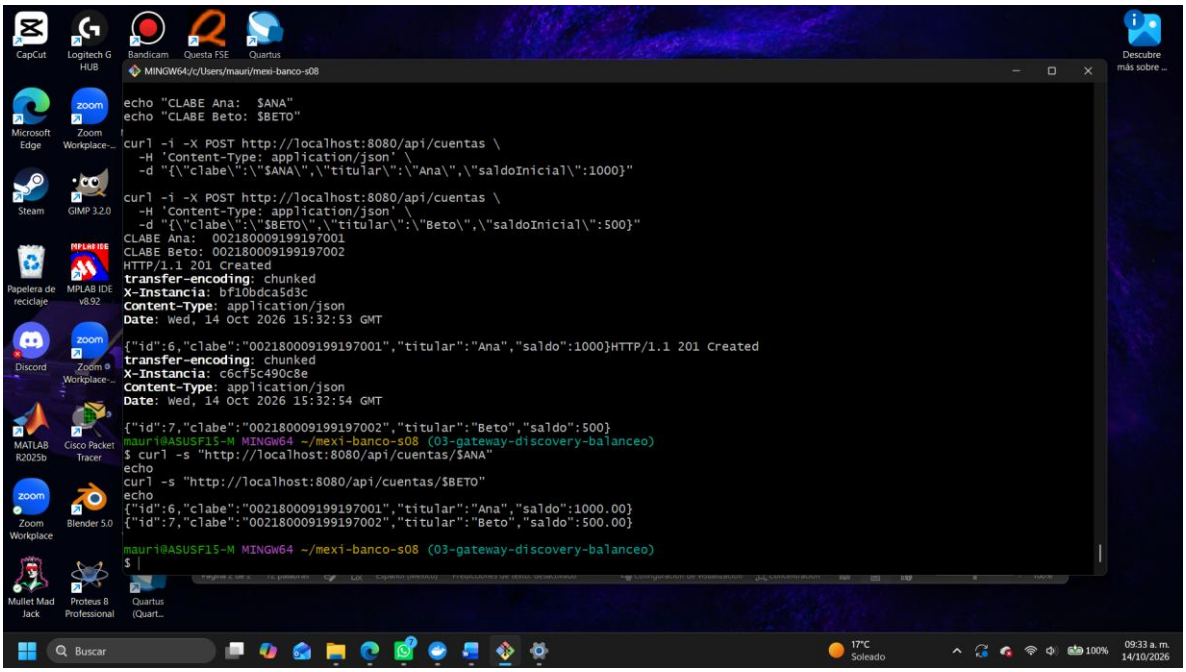
Al intentar construir me mandó un error, el cual investigando determinamos que se debe a los caracteres CR al final de los archivos mnw. Ejecutamos algunos comandos para corregirlos y ahora si se comienza a construir correctamente.



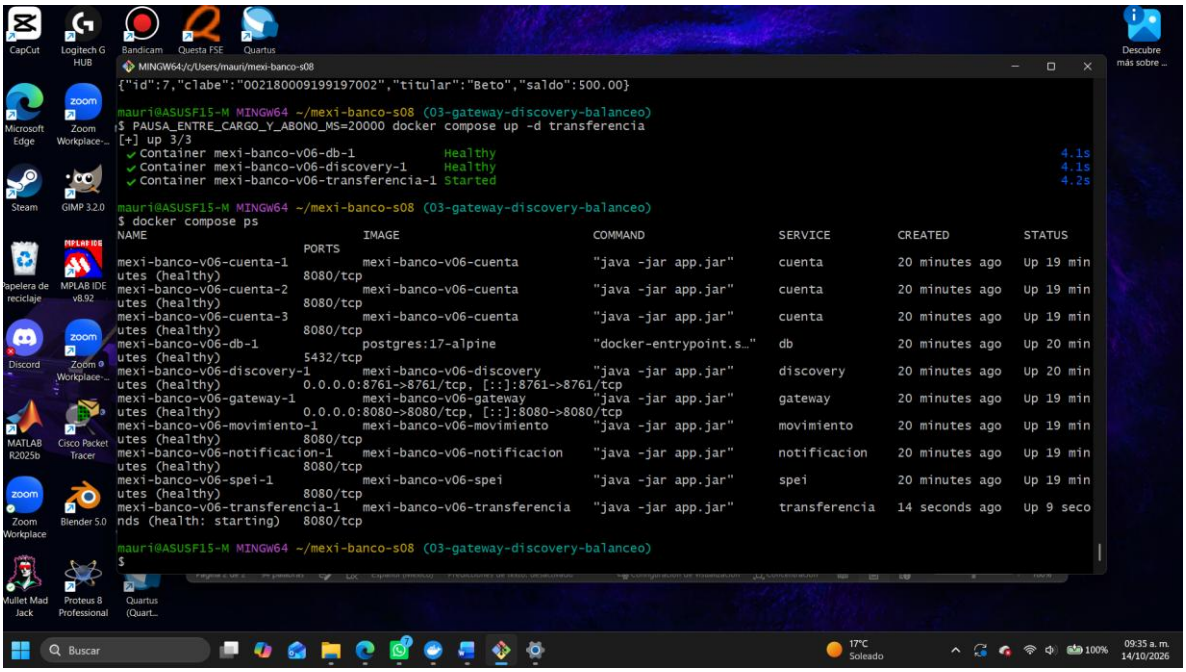
Esperamos a que todos digan healthy ejecutando Docker compose ps.



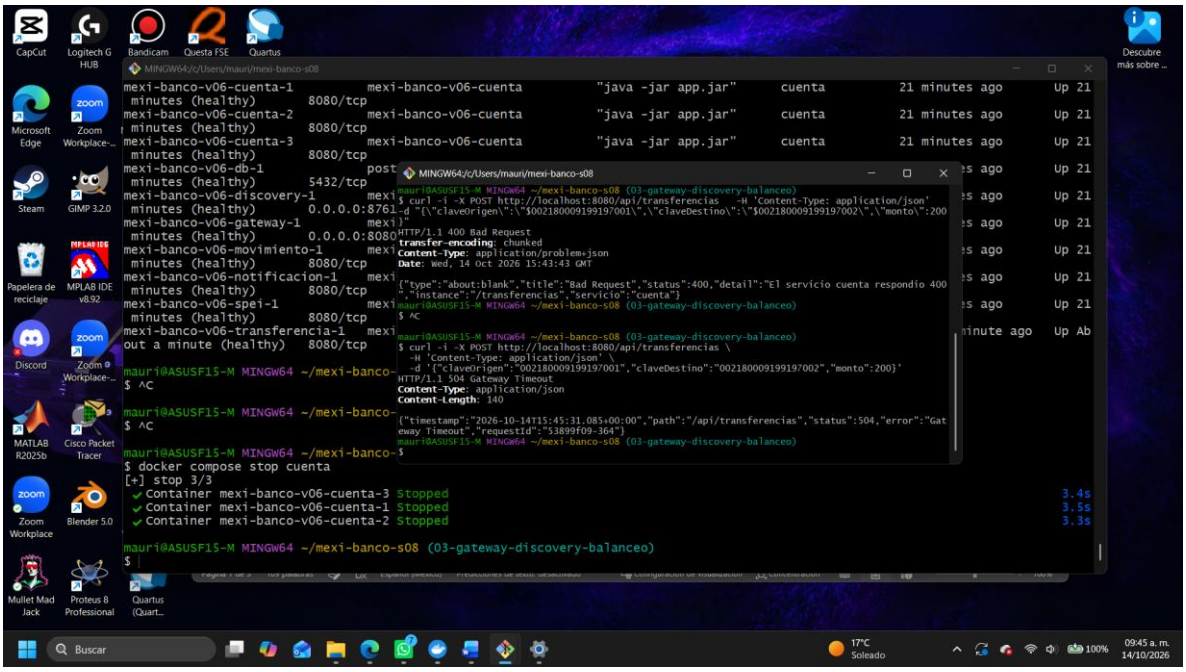
No existían cuentas creadas por defecto, por ende, nos disponemos a cerrar las cuentas de Ana y Beto a modo de prueba:



Creamos pausa entre cargo ya abono. Esperamos a que el estado healthy vuelva a aparecer.

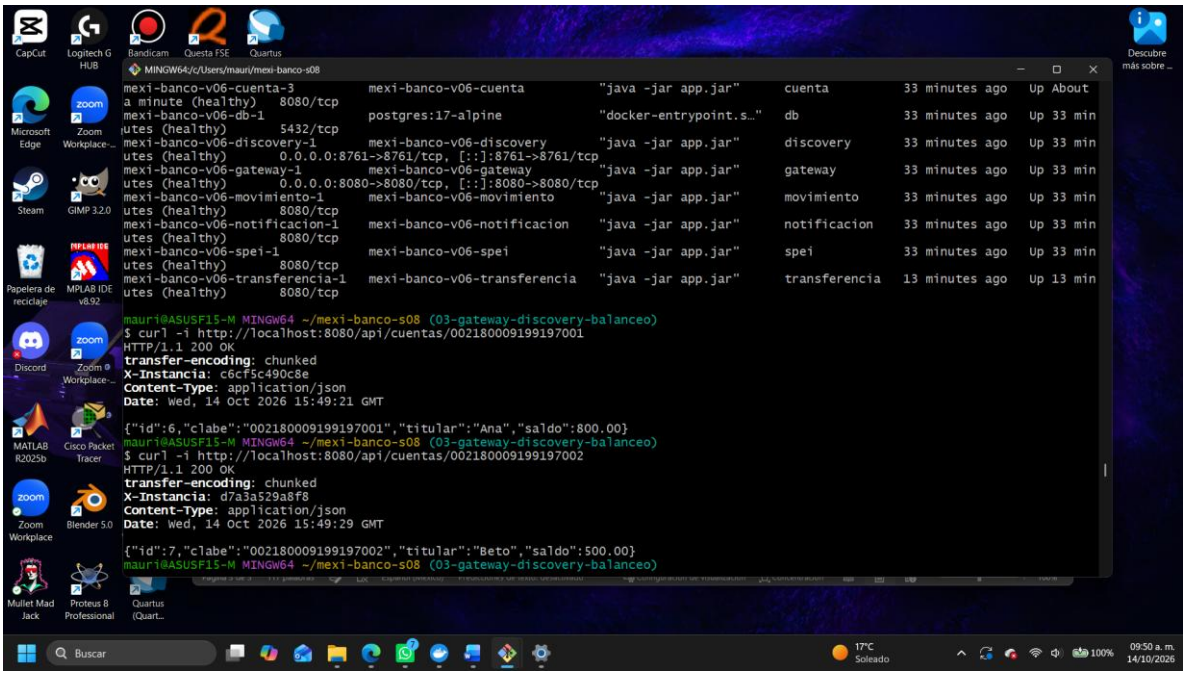


Lanzamos una transferencia de 200 y apagamos cuenta.



Tras esperar a que cuenta alcanzara el estado “healthy”, consultamos saldos de ambas cuentas. A la cuenta Ana se le descontaron los 200 de la transferencia, pero no le

llegaron a beto. El método de transferencia muestra no tener transferencia, porque no incorpora ningún mecanismo de seguridad ante posibles interrupciones del servicio, ejecutando cada una de las acciones por separado sin un backup para restaurarlas o la segmentación de las mismas en pasos para realizarlos de forma ordenada y verificar que cada uno se cumpla. A la cuenta de Ana se le descuentan los 200 de transferencia, pero si se interrumpe la operación completa, no existe una operación que deshaga este cambio ni un backup del que pueda apoyarse para una restauración.



Parte 1 · La saga orquestada (obligatoria) 14/10/2026 – 10am.

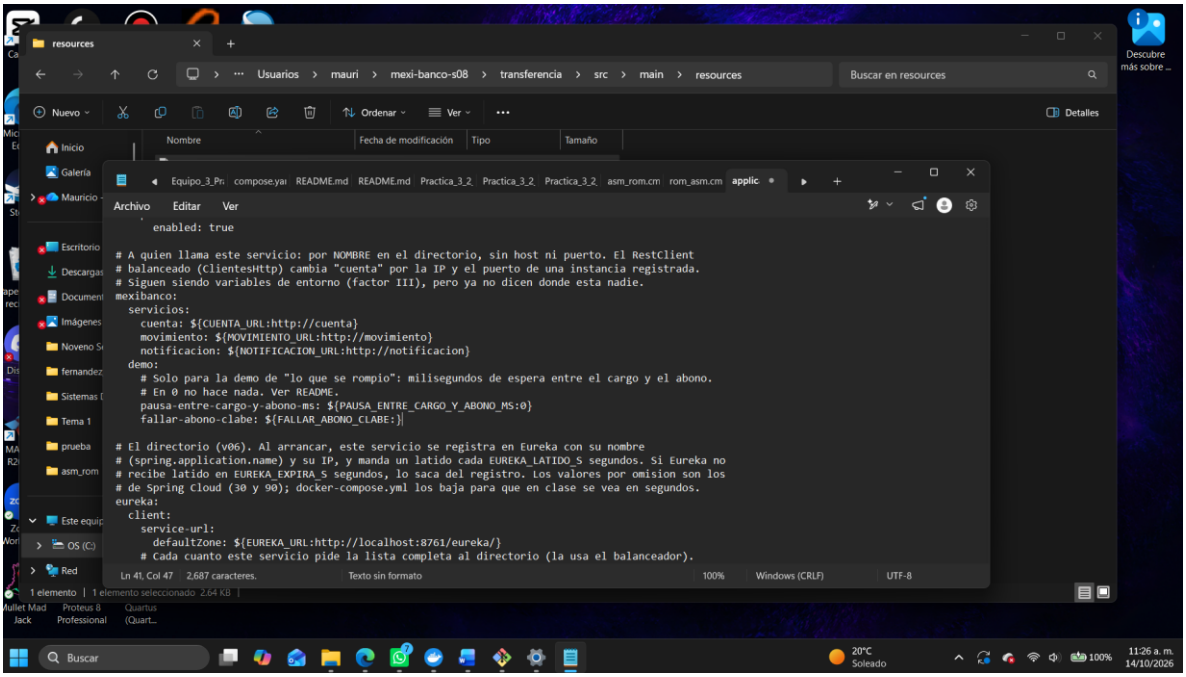
Para implementar la saga de transferencias se agregaron estados persistentes a la entidad Transferencia. Esto permite identificar en qué etapa se encuentra cada operación y conocer si terminó correctamente o si requiere una compensación.

Los estados incorporados fueron INICIADA, CARGADA, ABONADA, COMPLETADA, COMPENSADA y COMPENSACION_PENDIENTE. Con ellos se puede distinguir entre una transferencia en proceso, una operación finalizada y una transferencia que necesita recuperar el dinero debido a un fallo.

También se modificó la lógica del servicio de transferencias para actualizar el estado conforme avanza la operación. Primero se registra la transferencia, después se intenta retirar el dinero de la cuenta de origen y posteriormente se realiza el abono en la cuenta

de destino. Si ambas operaciones terminan correctamente, la transferencia se marca como COMPLETADA.

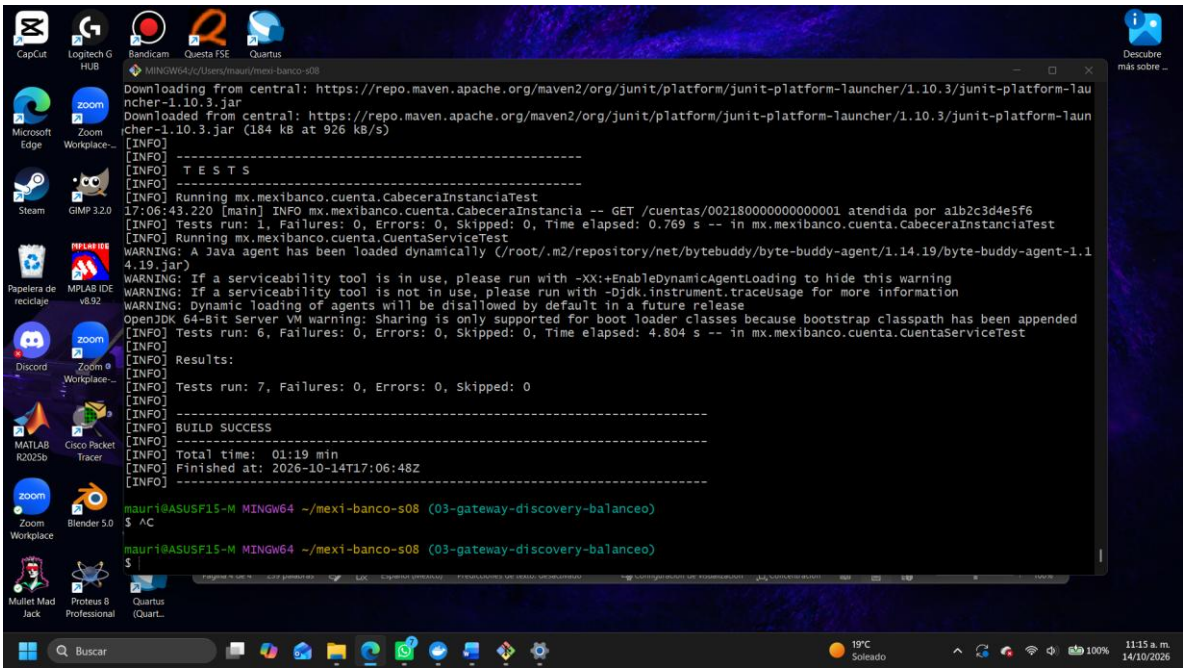
Cuando el abono falla, el servicio intenta devolver el importe a la cuenta de origen. Si la devolución se completa, el estado cambia a COMPENSADA. En caso de que también falle la devolución, se registra COMPENSACION_PENDIENTE, lo que permite identificar las operaciones que requieren atención y reintentar la compensación posteriormente.



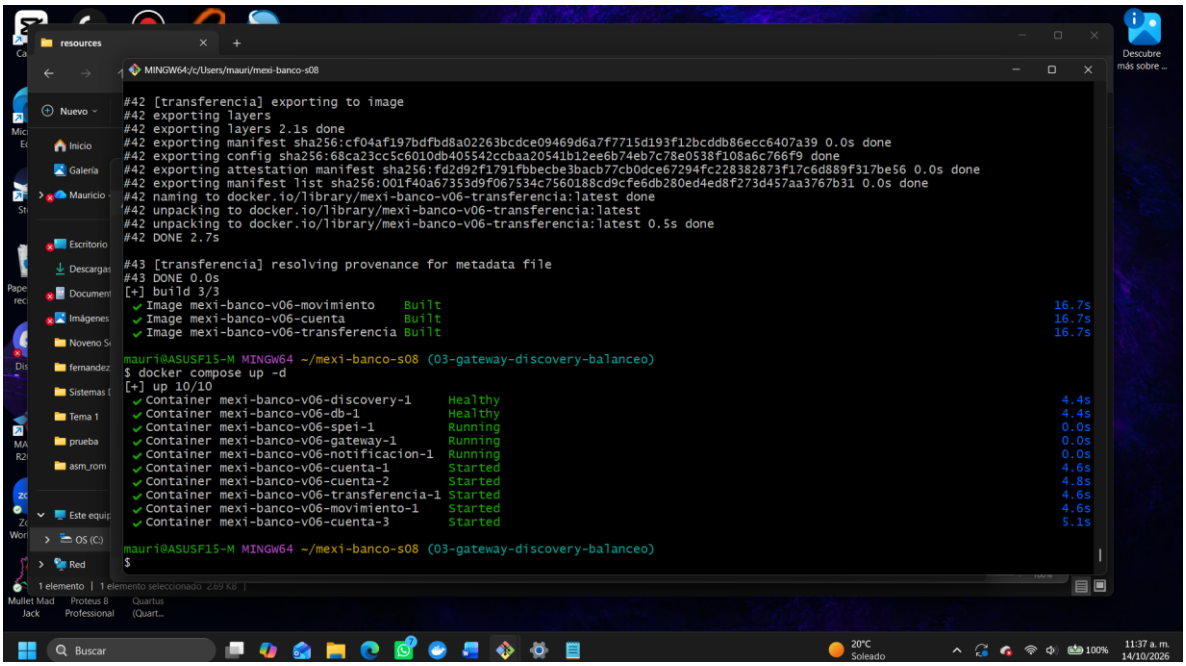
Además, se incorporó un endpoint para reintentar una compensación pendiente mediante una solicitud HTTP. La operación utiliza una clave de idempotencia asociada al identificador de la transferencia, con el propósito de evitar que un mismo reintento genere devoluciones duplicadas.

2. Preparación del entorno y pruebas automatizadas

Debido a que Java no se encuentra configurado directamente en Windows, las pruebas automatizadas se ejecutaron mediante un contenedor temporal con Java 21. Maven finalizó con el resultado BUILD SUCCESS, lo que confirma que la compilación y las pruebas ejecutadas en esa ejecución terminaron correctamente.

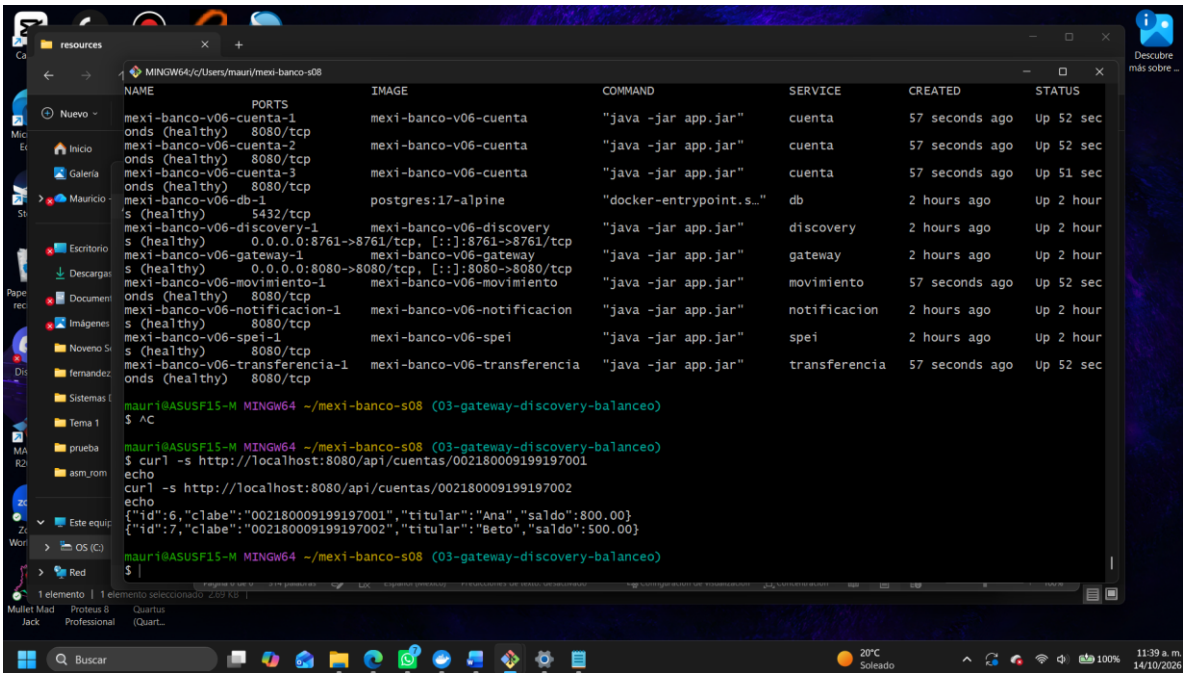


Posteriormente, se reconstruyeron los contenedores de los servicios modificados y se levantó nuevamente el entorno mediante Docker Compose para realizar las pruebas de integración.



3. Provocación de un fallo en el abono

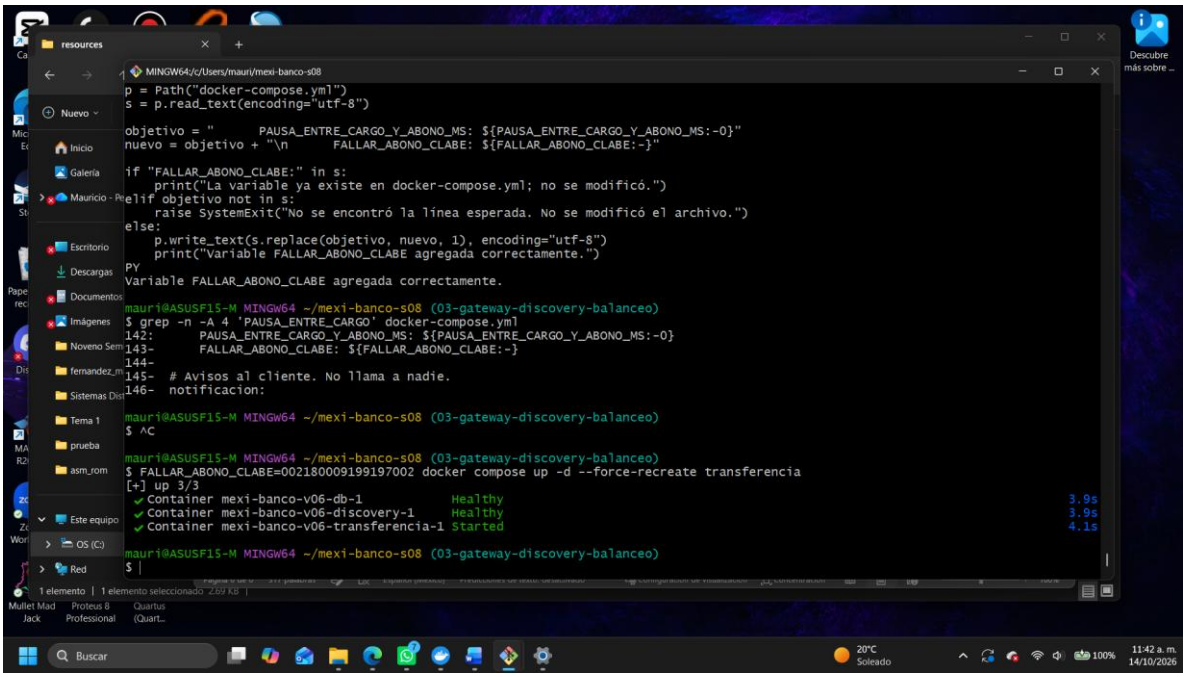
Antes de realizar la prueba se consultaron los saldos de las cuentas involucradas. Ana tenía $800.00 y Beto tenía $500.00.



Para probar el mecanismo de compensación, se configuró el servicio de transferencias para provocar un fallo controlado cuando el destino fuera la CLABE de Beto. Esta configuración permitió probar el escenario de error sin tener que detener los demás servicios del sistema.

Se solicitó una transferencia de $200.00 desde la cuenta de Ana hacia la cuenta de Beto. El retiro se realizó, pero el abono falló por la condición configurada. El servicio intentó compensar la operación; sin embargo, la devolución también falló debido a una restricción de la tabla de movimientos que no admitía el nuevo tipo de movimiento COMPENSACION_TRANSFERENCIA.

Como no fue posible confirmar la devolución, la transferencia quedó en estado COMPENSACION_PENDIENTE. Este estado indica que la operación no se completó y que todavía debe resolverse la devolución del dinero.

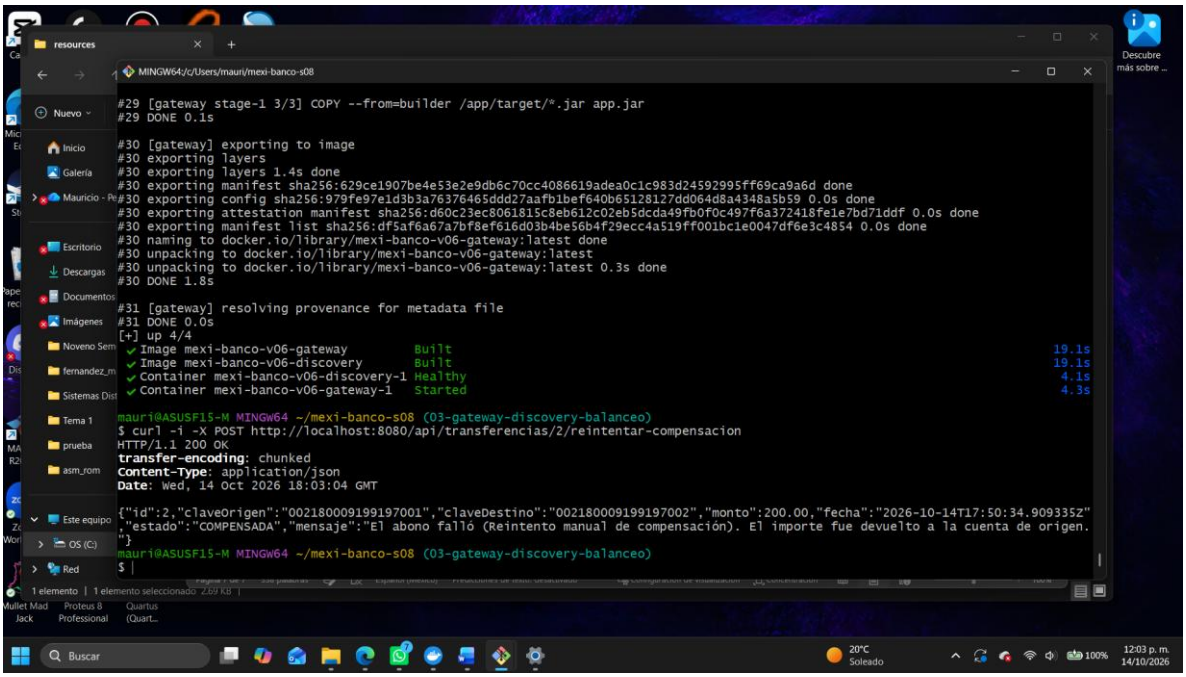


4. Reintento de la compensación

Después de identificar el problema, se corrigió la restricción de la base de datos para permitir el tipo de movimiento correspondiente a la compensación. También se ajustó la ruta del gateway para que el endpoint de reintento pudiera recibir las solicitudes.

Una vez corregidos ambos problemas, se ejecutó el endpoint de reintento para la transferencia identificada con el número 2. El servicio devolvió una respuesta HTTP 200 y registró el estado COMPENSADA, indicando que el importe había sido devuelto a la cuenta de origen.

Finalmente, se consultaron nuevamente los saldos de ambas cuentas. Ana terminó con $800.00 y Beto conservó $500.00. Por lo tanto, en este escenario el dinero retirado durante el intento fallido fue devuelto a la cuenta de origen, sin que se acreditara indebidamente a la cuenta de destino.

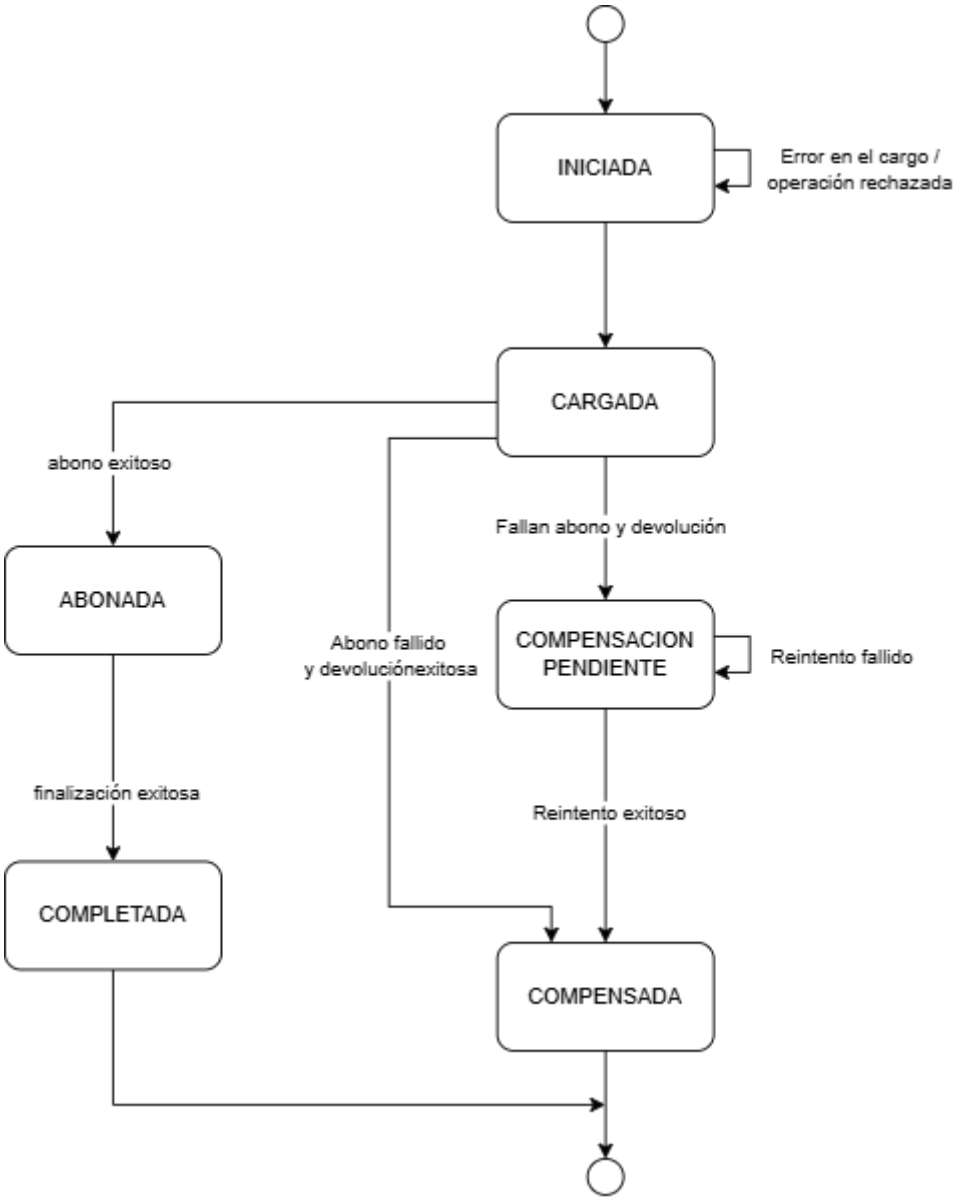


5. Diagrama de estados de la transferencia

El flujo de estados implementado se representa de la siguiente manera:

- INICIADA: la transferencia fue registrada.
- CARGADA: el dinero fue retirado de la cuenta de origen.
- ABONADA: el dinero fue acreditado a la cuenta de destino.
- COMPLETADA: la transferencia terminó correctamente.
- COMPENSADA: el abono falló y la devolución se completó.
- COMPENSACION_PENDIENTE: el abono falló y la devolución no pudo completarse; se requiere un reintento.

El siguiente diagrama resume las transiciones principales y los dos posibles resultados del proceso de compensación.



6. ¿Qué ocurre si la compensación también falla?

Si el abono a la cuenta de destino falla y tampoco es posible devolver el dinero a la cuenta de origen, la transferencia queda en estado COMPENSACION_PENDIENTE. Este estado evita considerar la operación como terminada y permite identificar que todavía existe una devolución por resolver.

Para terminarla, primero se debe corregir la causa del fallo y después ejecutar el endpoint de reintento de compensación. Si la devolución se realiza correctamente, el estado cambia a COMPENSADA. Si vuelve a fallar, permanece pendiente hasta que se pueda resolver.

Este mecanismo permite conservar evidencia del fallo, distinguir una operación pendiente de una compensada y realizar un reintento controlado sin iniciar una nueva transferencia.

Código 1. Definición de los estados

Archivo: EstadoTransferencia.java

```
public enum EstadoTransferencia {
  INICIADA,
  CARGADA,
  ABONADA,
  COMPLETADA,
  COMPENSADA,
  COMPENSACION_PENDIENTE
}
```

Se definieron los estados que permiten identificar la etapa de una transferencia. Los estados COMPLETADA y COMPENSADA representan los dos resultados principales de una operación, mientras que COMPENSACION_PENDIENTE permite identificar aquellas en las que la devolución del dinero todavía no se ha resuelto.

Código 2. Compensación y registro del resultado

Archivo: TransferenciaService.java

```
private Transferencia compensar(
    Transferencia transferencia, RuntimeException causa) {
  try {
    cuentas.compensar(
        transferencia.getClaveOrigen(),
        transferencia.getMonto(),
        "COMPENSACION-TRANSFERENCIA-" + transferencia.getId());

    transferencia.actualizarEstado(
        EstadoTransferencia.COMPENSADA,
        "El abono falló (" + mensajeDe(causa)
            + "). El importe fue devuelto a la cuenta de origen.");
  } catch (RuntimeException errorCompensacion) {
    transferencia.actualizarEstado(
        EstadoTransferencia.COMPENSACION_PENDIENTE,
```

```
            "El abono falló (" + mensajeDe(causa)
                + ") y no se pudo confirmar la devolución ("
                + mensajeDe(errorCompensacion)
                + "). Se requiere reintentar la compensación.");
    }

    return transferencias.save(transferencia);
}
```

El método compensar intenta devolver el importe a la cuenta de origen cuando falla el abono. Si la devolución se completa, registra el estado COMPENSADA; si también falla, guarda COMPENSACION_PENDIENTE junto con el motivo del error. Finalmente, persiste el estado actualizado de la transferencia.

Código 3. Reintento controlado

Archivo: TransferenciaService.java

```
public Transferencia reintentarCompensacion(Long id) {
    Transferencia transferencia = consultar(id);

     if (transferencia.getEstado() !=
EstadoTransferencia.COMPENSACION_PENDIENTE) {
          throw new ReglaDeNegocioException(
               "La transferencia no tiene una compensación pendiente");
      }

      return compensar(transferencia,
          new IllegalStateException("Reintento manual de compensación"));
```

El método reintentarCompensacion consulta la transferencia y verifica que su estado sea COMPENSACION_PENDIENTE. Si cumple la condición, vuelve a ejecutar el método de compensación; de lo contrario, rechaza la solicitud. Esto permite recuperar operaciones fallidas sin iniciar una nueva transferencia.

Código 4. Endpoint para reintentar la compensación

Archivo: TransferenciaController.java

```
@PostMapping("/{id}/reintentar-compensacion")
```

```
public Transferencia reintentarCompensacion(@PathVariable Long id) {
  return transferencias.reintentarCompensacion(id);
}
```

Se agregó un endpoint HTTP POST que recibe el identificador de una transferencia y solicita reintentar su compensación. El controlador delega la operación al servicio, donde se valida que la transferencia tenga una devolución pendiente.

Parte 2 · Circuit breaker en las llamadas a cuenta 14/10/2026 – 2:55pm

Creamos una clase independiente para que los tres estados, el contador y el tiempo de espera estén concentrados en un solo lugar.

```
package mx.mexibanco.clientes;

import mx.mexibanco.compartido.ErrorRemotoException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.client.ResourceAccessException;

import java.util.concurrent.Callable;
import java.util.concurrent.atomic.AtomicBoolean;

public class CircuitBreakerManual {

  public enum Estado {
    CERRADO,
    ABIERTO,
    SEMIABIERTO
  }

  private static final Logger log =
      LoggerFactory.getLogger(CircuitBreakerManual.class);

  private final int umbralFallos;
  private final long esperaAbiertoMs;

  private Estado estado = Estado.CERRADO;
  private int fallosConsecutivos = 0;
  private long abiertoDesde = 0;
```

```
  private boolean pruebaEnCurso = false;

  public CircuitBreakerManual(int umbralFallos, long esperaAbiertoMs) {
    this.umbralFallos = umbralFallos;
    this.esperaAbiertoMs = esperaAbiertoMs;
  }

  public <T> T ejecutar(Callable<T> operacion) {
    boolean esPrueba = prepararLlamada();

    try {
      T resultado = operacion.call();
      registrarExito(esPrueba);
      return resultado;
    } catch (Exception error) {
      registrarFallo(error, esPrueba);

      if (error instanceof RuntimeException runtime) {
        throw runtime;
      }
      throw new RuntimeException(error);
    }
  }

  private synchronized boolean prepararLlamada() {
    if (estado == Estado.ABIERTO) {
      if (System.currentTimeMillis() - abiertoDesde < esperaAbiertoMs) {
        throw new CircuitoAbiertoException(
            "Circuit breaker ABIERTO: llamada a cuenta rechazada");
      }

      estado = Estado.SEMIABIERTO;
      pruebaEnCurso = true;
      log.warn("Circuit breaker de cuenta: ABIERTO -> SEMIABIERTO");
      return true;
    }

    if (estado == Estado.SEMIABIERTO) {
      if (pruebaEnCurso) {
        throw new CircuitoAbiertoException(
            "Circuit breaker SEMIABIERTO: prueba en curso");
      }

      pruebaEnCurso = true;
```

```
        return true;
      }

      return false;
    }

    private synchronized void registrarExito(boolean esPrueba) {
      fallosConsecutivos = 0;

      if (estado == Estado.SEMIABIERTO && esPrueba) {
        estado = Estado.CERRADO;
        pruebaEnCurso = false;
        log.info("Circuit breaker de cuenta: SEMIABIERTO -> CERRADO");
      }
    }

    private synchronized void registrarFallo(Exception error, boolean esPrueba) {
      if (!esFallaDeDisponibilidad(error)) {
        if (esPrueba) {
          estado = Estado.ABIERTO;
          abiertoDesde = System.currentTimeMillis();
          pruebaEnCurso = false;
          log.warn("La prueba de cuenta no confirmó recuperación; circuito
ABIERTO");
        }
        return;
      }

      if (esPrueba || estado == Estado.SEMIABIERTO) {
        estado = Estado.ABIERTO;
        abiertoDesde = System.currentTimeMillis();
        fallosConsecutivos = umbralFallos;
        pruebaEnCurso = false;
        log.warn("Circuit breaker de cuenta: SEMIABIERTO -> ABIERTO");
        return;
      }

      fallosConsecutivos++;

      if (fallosConsecutivos >= umbralFallos) {
        estado = Estado.ABIERTO;
        abiertoDesde = System.currentTimeMillis();
        pruebaEnCurso = false;
        log.warn("Circuit breaker de cuenta: CERRADO -> ABIERTO");
```

```
    }
  }

  private boolean esFallaDeDisponibilidad(Exception error) {
    if (error instanceof ErrorRemotoException remoto) {
      return remoto.getEstado() >= 500;
    }

    return error instanceof ResourceAccessException;
  }

  public synchronized Estado getEstado() {
    return estado;
  }

  public static class CircuitoAbiertoException extends RuntimeException {
    public CircuitoAbiertoException(String mensaje) {
      super(mensaje);
    }
  }
}
```

Integramos el circuito en CuentaCliente. En CuentaCliente.java hicimos un cambio bastante localizado: agregamos el circuit breaker como atributo y envolvimos las cuatro llamadas a cuenta con circuito.ejecutar(...).

Debajo de private final RestClient http;, añadimos:

```
private final CircuitBreakerManual circuito =
    new CircuitBreakerManual(3, 10_000);
```

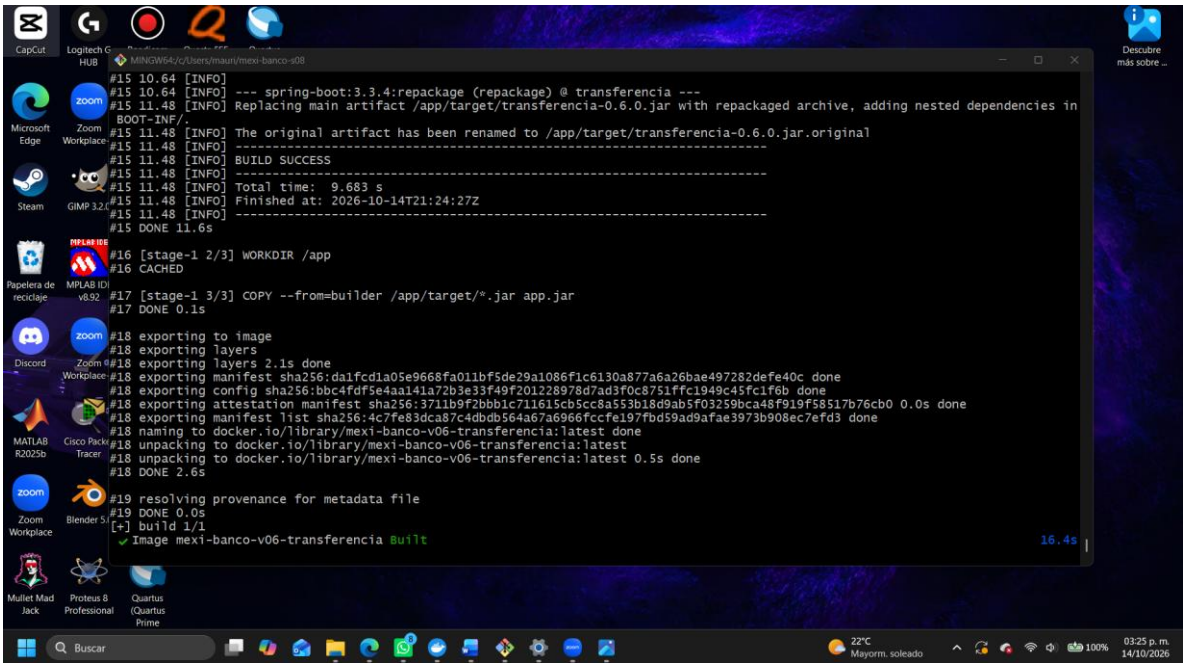
Esto configura el circuito para abrirse después de 3 fallos consecutivos de disponibilidad y esperar 10 segundos antes de permitir una prueba de recuperación.

Igualmente, en cada método, hicimos que la llamada HTTP que antes se ejecutaba directamente ahora está dentro de circuito.ejecutar(() -> ...).

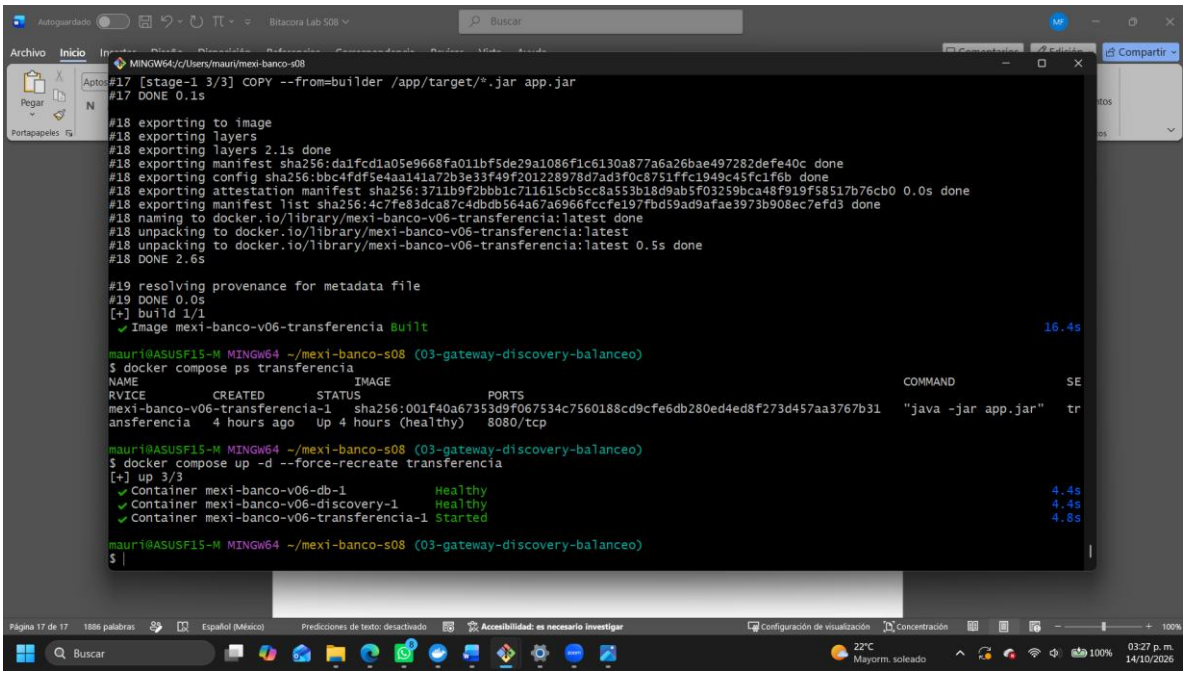
El mismo cambio se aplicó a cargar, abonar y compensar.

- cargar: la llamada a http.post() que usa /cuentas/{clabe}/cargos.
- abonar: la llamada a http.post() que usa /cuentas/{clabe}/abonos.
- compensar: la llamada a http.post() que usa /cuentas/{clabe}/compensaciones.

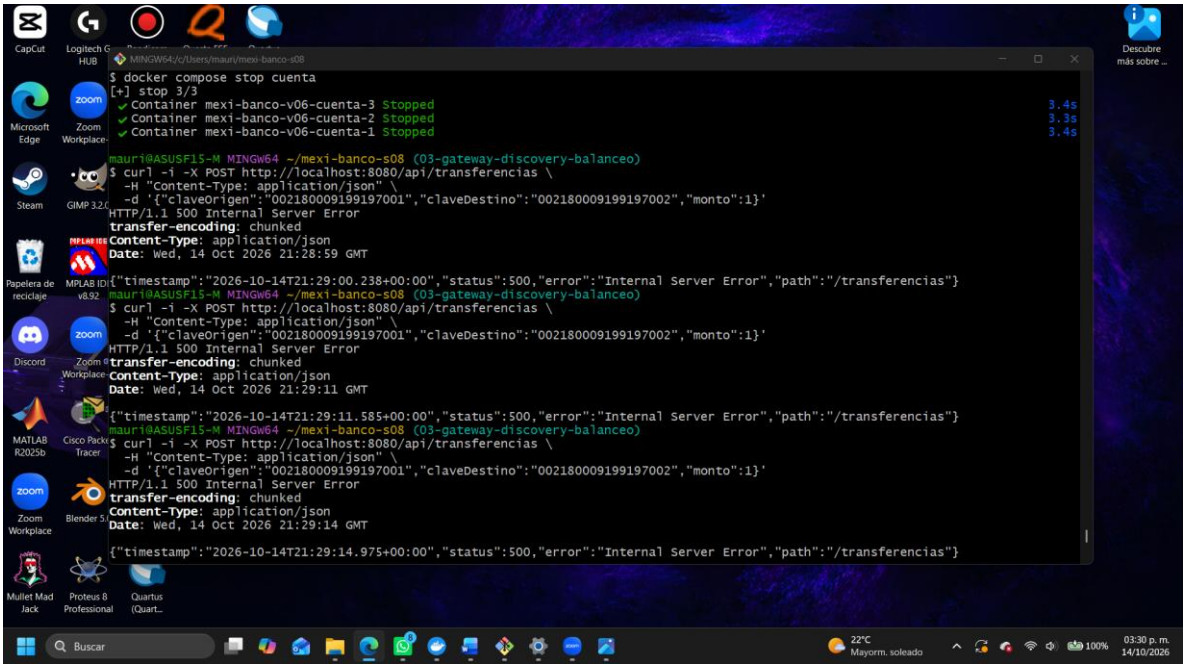
Después de hacer esas modificaciones, volemos a construir la imagen de transferencia.



Volvemos a crear el contenedor con la nueva imagen que nos permita probar el circuit breaker.



Detenemos el servicio de cuenta y hacemos algunas peticiones de prueba.



Vemos que CuentaCliente está usando CircuitBreakerManual, pero no aparece el mensaje CERRADO -> ABIERTO. El error real es: No instances available for cuenta.

Ajustamos la detección de fallos en **transferencia/src/main/java/mx/mexibanco/clientes/CircuitBreakerManual.java**

Ingresamos el siguiente método. Este hace que el circuito también reconozca cuando Eureka/LoadBalancer no encuentra instancias disponibles, sin contar indiscriminadamente cualquier IllegalStateException.

```
private boolean esFallaDeDisponibilidad(Exception error) {
    if (error instanceof ErrorRemotoException remoto) {
        return remoto.getEstado() >= 500;
    }

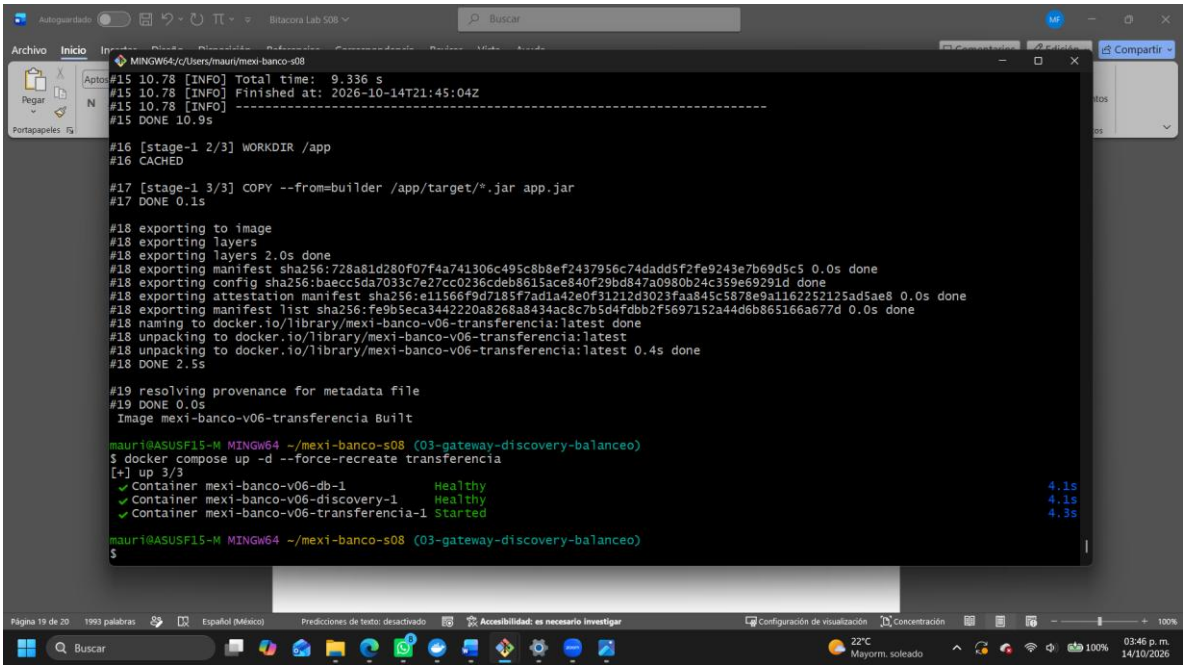
    if (error instanceof ResourceAccessException) {
        return true;
    }

    if (error instanceof IllegalStateException) {
        return error.getMessage() != null
                && error.getMessage().contains("No instances available");
    }

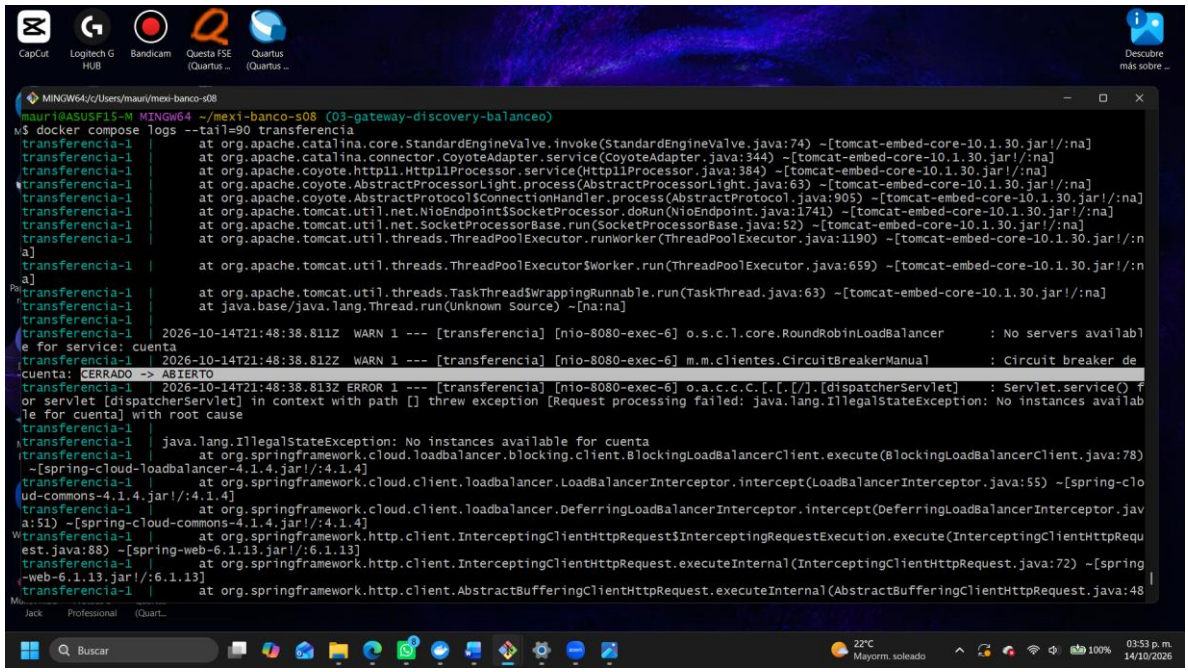
    return false;
```

```
}
```

Reconstruimos la imagen.

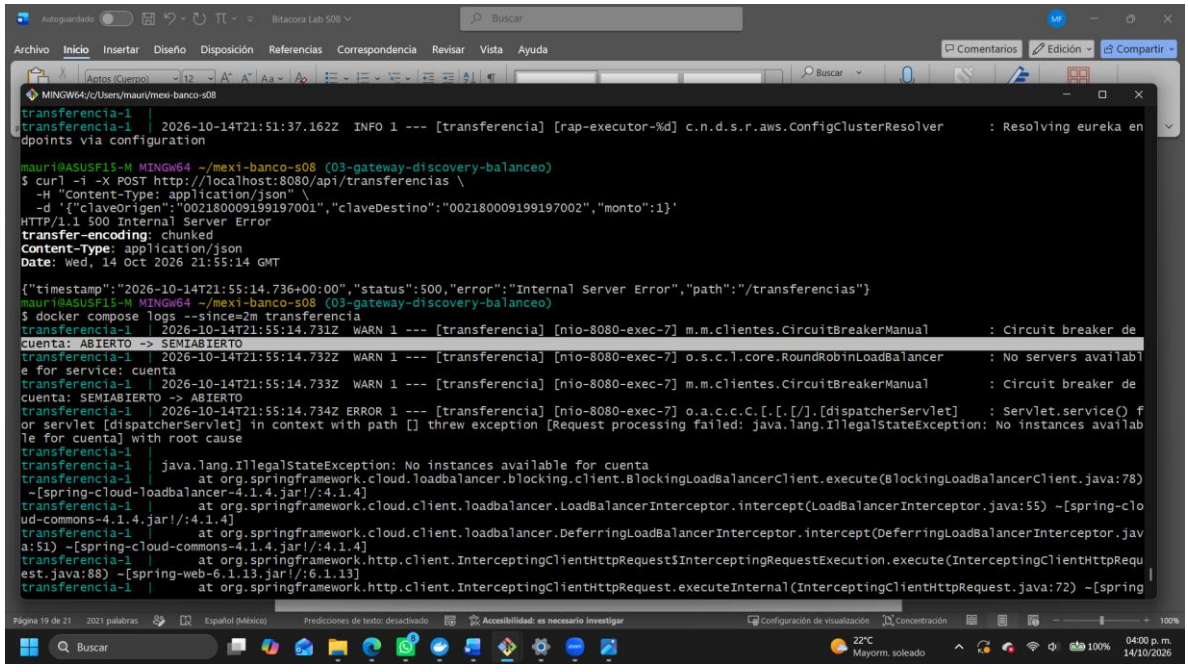


Tras provocar 3 fallos consecutivos para llegar al umbrak de tres fallos, consultamos los registros de transferencia para verificar que el circuit beaker pasó de cerrado a abierto.



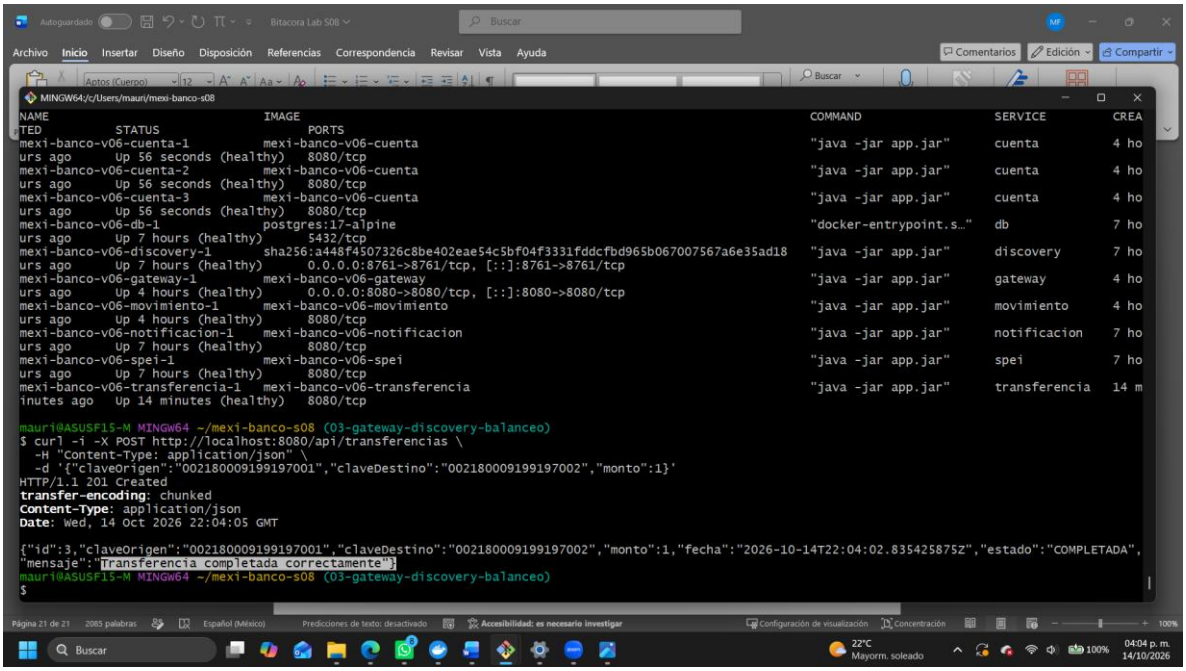
Encontramos un detalle importante al consultar los logs de transferencia nuevamente. El circuito pasó automáticamente de ABIERTO a SEMIABIERTO, intentó contactar a cuenta y volvió a ABIERTO porque el servicio seguía detenido.

Eso significa que la transición a semiabierto funciona.

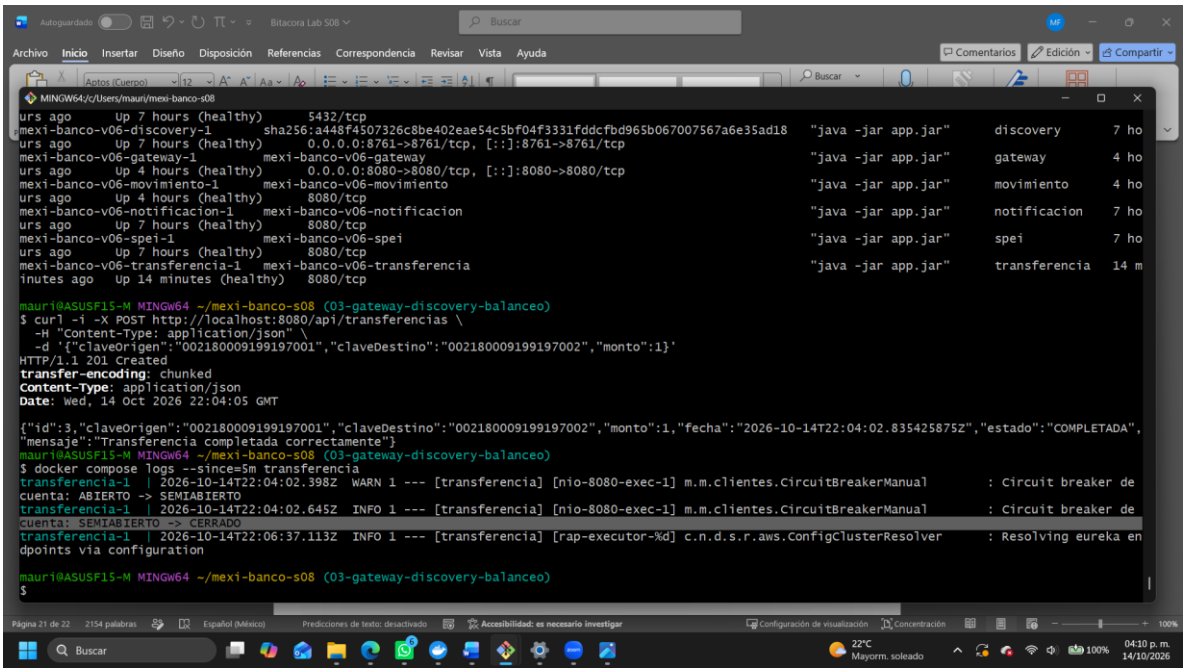


Nos queda probar el rechazo de llamadas mientras el circuito permanece abierto. Volvemos a reiniciar cuenta y esperamos a que los tres marquen healthy.

Creando otra transferencia de prueba, obtuvimos HTTP 201 Created y quedó en estado COMPLETADA. Eso confirma que, al volver a estar disponible cuenta, el flujo de transferencias puede funcionar de nuevo.



Consultando los logs, vemos que el circuito paso de Abieto a Semiabierto, y finalmente a Cerrado. De esta forma comprobamos que el circuit breaker manual funciona en los tres estados y que se recupera cuando vuelve el servicio.



Aislamiento y consistencia durante la saga

La implementación de la saga nos permite coordinar la transferencia y ejecutar operaciones compensatorias cuando una etapa falla, pero no proporciona aislamiento de las transacciones entre los distintos servicios. Por ejemplo, después de realizar el cargo en la cuenta de origen y antes de completar el abono en la cuenta de destino, una consulta independiente al saldo de origen puede observar temporalmente el saldo reducido. Si el abono falla, ese saldo permanece reducido hasta que la compensación se ejecute correctamente. Durante ese tiempo, otras operaciones podrían verse afectadas por la disponibilidad temporal de los fondos. Por lo tanto, la saga ofrece un mecanismo de recuperación ante posibles fallos, pero no garantiza que todos los participantes observen una única transacción atómica. Para controlar esta situación podrían utilizarse estrategias adicionales, como registrar operaciones pendientes, mantener un historial de movimientos o reservar fondos mientras se completa la transferencia.